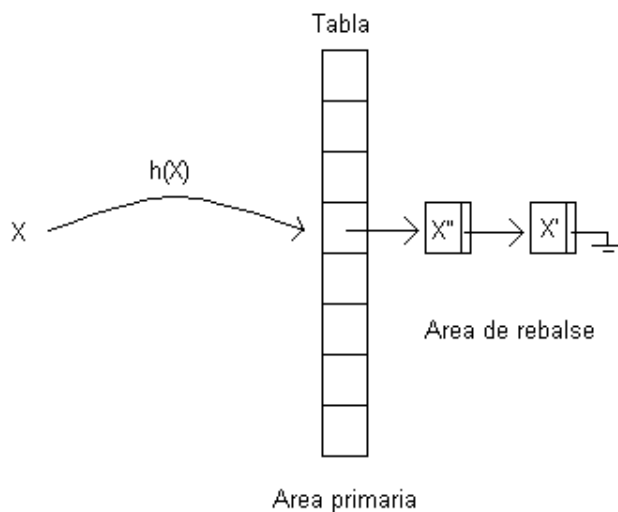


CC3001 Algoritmos y Estructuras de Datos**Profesores:** Nelson Baloian, Patricio Poblete, e Iván Sipirán**Auxiliares:** Sebastián Acuña, Daniela Espinoza, Cristián Llull Torres, Benjamín Osses, Anish Samtani, y Marcelo Zamorano**Auxiliar 9: Ordenación pt.2**

07 de noviembre de 2025

P1. Hashing Con Encadenamiento

Suponga que se tiene una tabla de hashing con encadenamiento, de tamaño 10, inicialmente vacía, con la función de hashing $h(x) = (13 \cdot x) \bmod 10$ (por ejemplo, $h(64) = 2$).

Muestre en la siguiente tabla el resultado de insertar (a mano) la siguiente secuencia de llaves: 34, 59, 45, 27, 14, 22, 75, 25

0	1	2	3	4	5	6	7	8	9

P2. Yaroslavskiy

Quicksort se hizo popular poco después de su presentación original por Hoare, muchos autores contribuyeron con variaciones y trucos de implementación para el algoritmo básico. Después de las mejoras a Quicksort, en la década de 1990, casi todas las bibliotecas de programación usaban versiones casi idénticas del algoritmo: la implementación clásica de Quicksort se había estandarizado.

No fue hasta 2009, más de una década más tarde, que un desarrollador ruso previamente desconocido, Владимир Ярославский (*Vladimir Yaroslavskiy*), generó un cambio radical al publicar el resultado de sus experimentos al público: una implementación de Quicksort de doble pivote que supera claramente al clásico Quicksort de Java 6 de Oracle.

La principal innovación es la partición ternaria (en vez de la binaria) que se puede ver en la representación de la figura más abajo.

El hallazgo de Yaroslavskiy fue tan sorprendente que la gente inicialmente se mostró reacia a creerlo, pero su Quicksort finalmente fue instalado en millones de dispositivos con el lanzamiento de Java 7 en 2011.

Implemente el algoritmo de ordenación propuesto por Yaroslavskiy. Su diferencia con el Quicksort usual es que utiliza dos pivotes en vez de uno. Compare los resultados con el quicksort original

P1	< P1	P1 <= & <= P2	?	> P2	P2
left		L	K	G	right
	part I	part II	part IV	part III	

```

QuicksortYaro(A, left, right)
  P1 = A[left]; P2 = A[right];
  L = left + 1; K = L ;
  G = right - 1;
  while K <= G:
    if A[K] < P1:
      Swap A[K] and A[L]
      L = L + 1
    else
      if A[K] > P2:
        while A[G] > P2 and K < G:
          G = G - 1
        end while
        Swap A[K] and A[G]
        G = G - 1
        if A[K] < P1:
          Swap A[K] and A[L]
          L = L + 1
        end if
      end if
    end if
    K = K + 1
  end while
  L = L - 1 ; G = G + 1 ;
  Swap A[left] and A[L]
  Swap A[right] and A[G]
  QuicksortYaro(A, left, L - 1)
  QuicksortYaro(A, L + 1, G - 1)
  QuicksortYaro(A, G + 1, right)

```

P3. Quickselect [P1 C2 2024-2]

Dado un arreglo a de tamaño n , y dado un entero k ($0 \leq k \leq n$), se desea reordenar los datos para que los k menores elementos queden en las k primeras posiciones, ordenados de menor a mayor. Modifique el algoritmo de Quicksort para que realice esta tarea. *Sugerencia: Inspírese en Quickselect*